TECH MAHINDRA CODE - GRAND FINALE

Network Anomaly Root-Cause Assistant

Find the component that caused the outage, not the loudest victim. Deterministic, evidence-backed root-cause analysis constrained to the network's own topology.

- > Causation constrained to the network topology, not timestamp correlation
- > A deterministic decision core: the same incident, the same answer, every time
- > Bounded AI that explains and verifies, but never picks the cause
- > Agentic RAG: remediation grounded in retrieved NOC runbooks, with citations
- > A tamper-evident, SHA-256 hash-chained audit trail

TEAM - BUILT AS SIX ISOLATED LAYERS

Arsh	Data & Testbed
Isha	Anomaly Detection
Nishant	Data Contract & Causal Engine
Shruthika	Frontend / NOC Dashboard
Swayam	AI Agents, Agentic RAG, API & Audit

Executive summary

When a network breaks, the monitoring system does not raise one alert. It raises thousands, and most of them are symptoms, not causes. Our system reads the telemetry, finds the anomalies, and uses the network's own dependency map to name the single component that caused the outage.

The core insight is simple: a component can only be the root cause of a symptom if a real dependency path exists from it to that symptom. By constraining causal inference to the physical topology, we separate genuine causation from mere time-correlation, which is exactly what lets us blame the database that failed rather than the web server that merely looks worst.

In practice the engine asks the three questions a good human investigator would ask, but in milliseconds and across the whole estate at once. Can this component even reach the symptom through the dependency graph? Did it fail before the symptom did? And does it explain the other failures, or is it just one more victim? Only a component that passes all three earns the top of the ranking, and the reasoning behind every placement is kept as cited evidence rather than hidden inside a model.

What stands out

- Causation, not correlation. Suspects are physically restricted to components that can actually reach the symptom through the dependency graph. This is the moat.
- A deterministic decision engine. The same incident always produces the same ranking, and every decision is explainable from a cited evidence ledger. It is not a black box.
- AI used with discipline. Bounded OpenAI agents write the narrative, adversarially stress-test the verdict, and recommend fixes grounded in retrieved NOC runbooks (agentic RAG), but they never pick the cause and cannot override the ranking.
- A tamper-evident audit trail. Every step is written to a SHA-256 hash-chained ledger, verifiable on demand.
- It is a product, not a canned demo. Upload your own dataset and the same engine analyses it, or returns a clean, explained healthy verdict.

The one line to remember
Everyone else correlates by time and blames the loudest victim. We constrain causation to the physical dependency graph, decide the root cause deterministically, and use AI only to explain and verify, which is why our answer is both trustworthy and reproducible.

The problem

A network operations centre can receive thousands of alerts an hour. The hard part is not seeing that something is wrong. It is knowing which red light is the cause and which are downstream noise.

When a database saturates its connection pool, every service that depends on it slows down. The web tier times out, the application tier queues up, and dashboards turn red across the board. To the on-call engineer, the web server looks broken because it has the loudest alarms and the worst numbers. But the web server is a victim. The real culprit is the database, several hops upstream, quietly refusing new connections.

Tools that correlate events by timestamp routinely blame whichever component moved first or looks worst, which is frequently the victim. The engineer then spends the bulk of mean-time-to-resolution just deciding which signal is the true cause. That is the expensive problem we set out to remove.

The question we answer

Given a storm of anomalies across a network, which single component is the root cause, and how do we prove it rather than guess by timing?

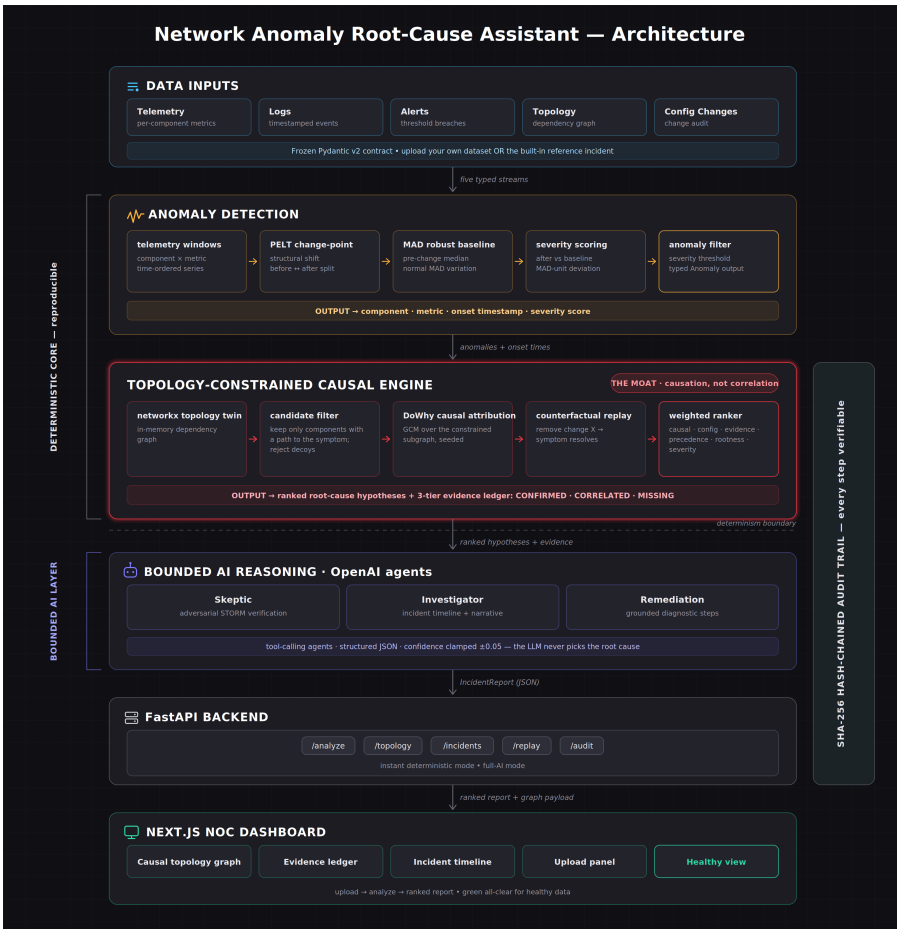
Our solution

The system ingests five typed data streams (telemetry, logs, alerts, topology, and configuration changes), detects anomalies together with the exact moment each one began, and then ranks the true root cause by walking the dependency graph. It is deliberately split into two layers.

> A deterministic core decides the root cause using robust statistics and graph rules. Same input, same answer, always, and every decision traces back to evidence.

> A bounded AI layer only does language work: the incident narrative, an adversarial second opinion, and grounded remediation steps. Its influence on confidence is clamped so it can never flip the ranking.

This is the vending-machine-over-slot-machine principle: determinism where correctness matters, AI where language matters. The full pipeline is shown below.



End-to-end architecture. Five data streams flow through detection and the topology-constrained causal engine (the deterministic core), then the bounded AI layer, the FastAPI backend, and the NOC dashboard, with a SHA-256 audit trail spanning every step.

How it works, end to end

- Ingest. An incident is a bundle of typed streams: a topology (components and dependency edges), telemetry (per-component metric windows), and optional logs, alerts, and configuration changes.
- Detect. A robust baseline and change-point detection flag which metrics are abnormal and, critically, the exact moment each anomaly began, because causation needs time order.
- Constrain. Candidate causes are filtered to components that have a real dependency path to the symptom. Everything else is rejected as a decoy, and the reason is shown to the user.
- Rank. A weighted model combines causal attribution, configuration-change proximity, evidence strength, temporal precedence, rootness, and severity into a single, explainable confidence score.
- Explain. Bounded agents write the incident narrative, adversarially stress-test the leading hypothesis, and translate the missing-evidence gaps into concrete remediation steps, retrieving the matching NOC runbooks from the knowledge base (agentic RAG) so every step cites the playbook behind it.
- Audit. Every step is appended to a hash-chained ledger, so the whole investigation can be replayed and verified for tampering after the fact.

Why we built it this way

Every major decision was made to maximise trust and reproducibility. Here is what we chose, and what we deliberately chose against.

Topology-constrained causation
instead of: timestamp correlation of alerts
A suspect must have a real dependency path to the symptom, or it is rejected as a decoy no matter how bad it looks. This single rule converts a correlation engine into a causation engine and defeats the blame-the-loudest-victim failure that plagues timestamp tools.

A deterministic decision core
instead of: asking an LLM 'what is the root cause?'
An LLM verdict is a slot machine: plausible, unaccountable, and different every run. Our ranking is pure math and graph traversal, so it is identical on every run and fully explainable from the evidence ledger. Judges can audit exactly why db-01 ranked first.

PELT change-point detection with a robust MAD baseline
instead of: static thresholds
Causation needs time order, so we must recover the onset of each anomaly, not just that a threshold was crossed. The baseline is measured only on the pre-onset segment and with the median absolute deviation, so a sustained fault cannot make itself look normal and heavy-tailed metrics cannot fool it.

An in-memory networkx topology twin
instead of: a graph database such as Neo4j
At 30 to a few hundred nodes, the impact-path and blast-radius queries run in microseconds with zero external services to operate. Neo4j earns its place only at much larger scale or for GraphRAG, so we keep it as a documented scale option rather than premature infrastructure.

Bounded, confidence-clamped AI
instead of: an autonomous agent that decides the outcome
The agents explain, adversarially verify (the STORM method), and recommend, but their influence on confidence is clamped to plus or minus 0.05, so they can sharpen the ranking yet never override the deterministic winner. Trust is guaranteed by construction, not by hoping the model behaves.

A weighted ranker that deliberately down-weights severity
instead of: ranking by alert severity
The victim usually looks the worst, so ranking by severity blames the victim. We put the majority of the score on causal attribution, what recently changed, and evidence strength, and give severity the smallest share, so the true cause wins even when a downstream component shows scarier numbers.

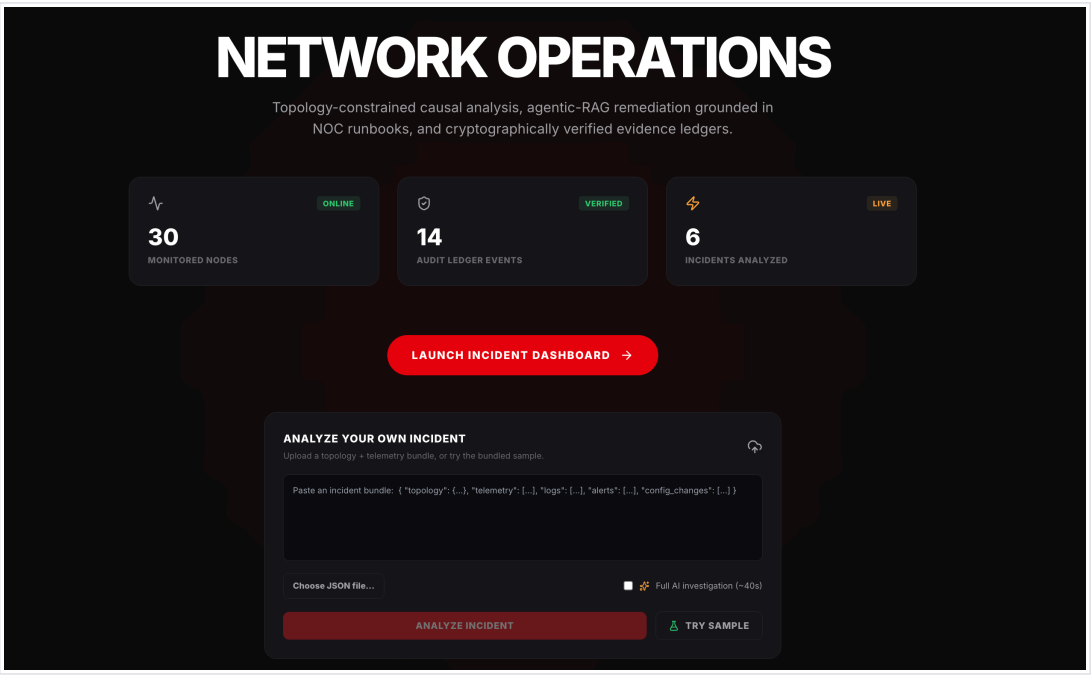
A frozen data contract with domain invariants
instead of: loose, untyped JSON between components
Evidence cannot be cited without a source record, and a hypothesis path must start at the named cause. These rules make dishonest or broken output impossible by construction, and the same contract drives both the Python backend and the TypeScript frontend so they never drift apart.

The product, live

A worked example: an uploaded incident where an application-tier capacity exhaustion propagates to the web tier. Every screen below is the running application, not a mock-up.

1. The operations console

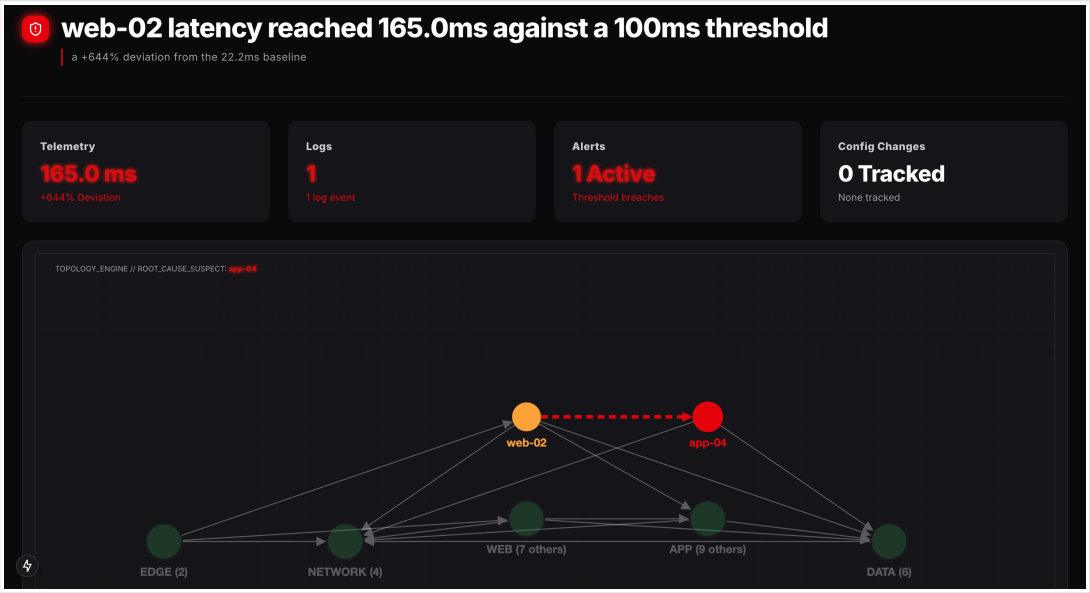
The landing view shows the monitored estate at a glance and an upload panel to analyse your own incident or the bundled sample. The counters are live: they climb as real analyses are run, and the audit ledger grows with every investigation.



The landing console: live estate and usage counters, the incident launcher, and bring-your-own-data upload.

2. The verdict, on the graph

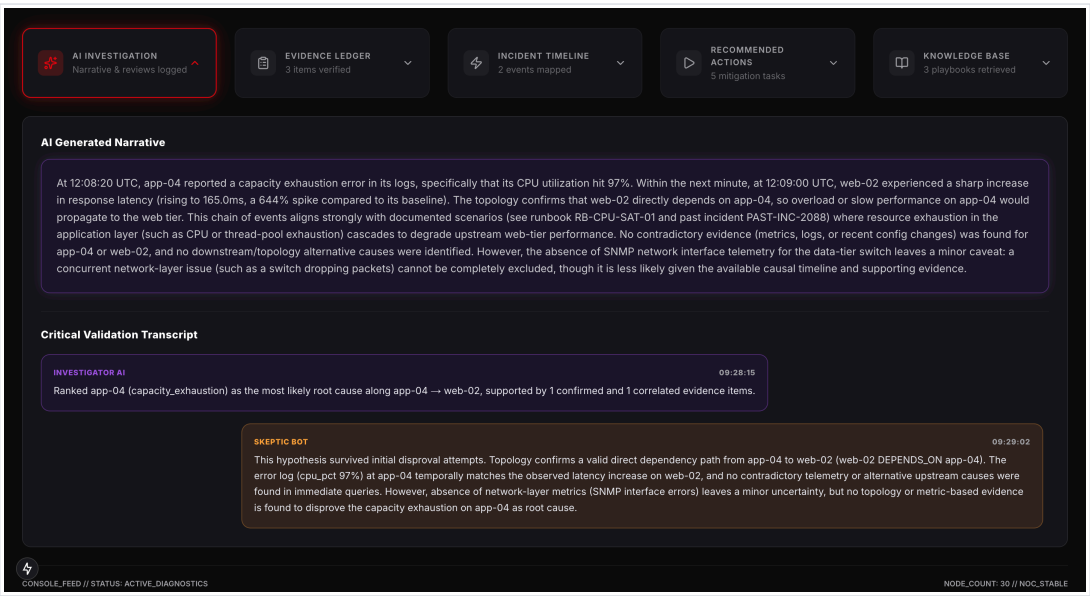
Opening the incident, the causal engine has already decided. The symptom (web-02, amber) shows a +644% latency deviation against its baseline, but the engine points past it, along the real dependency path, to the true root cause (app-04, red). Every healthy node stays green. This is causation, drawn.



The causation verdict: app-04 is named the root cause even though web-02 is the component showing the symptom. The red arrow is the propagation path along real dependencies.

3. The AI investigation, grounded and stress-tested

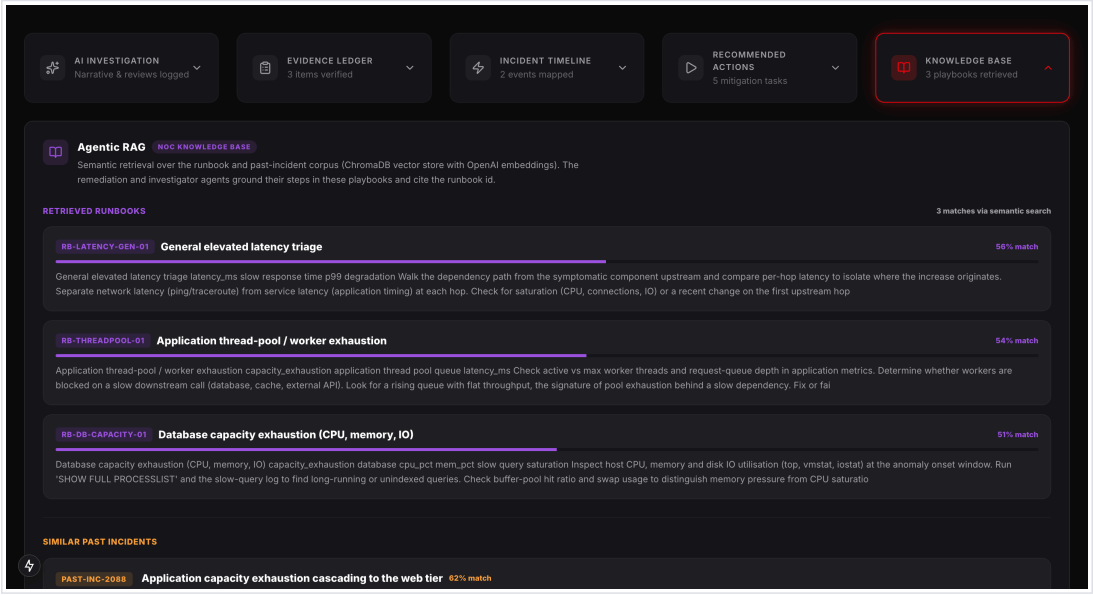
The investigator writes a plain-English narrative of what happened, and the skeptic agent adversarially tries to disprove it. Both are grounded: note that the narrative explicitly cites a retrieved runbook and a similar past incident, and the skeptic reports that the hypothesis survived disproval attempts while honestly flagging the one remaining uncertainty.



The AI narrative cites its sources (runbook RB-CPU-SAT-01 and past incident PAST-INC-2088), and the skeptic transcript records the adversarial verification verdict.

4. Agentic RAG: the NOC knowledge base

The Knowledge Base panel exposes the retrieval layer directly. Each entry is a real NOC playbook pulled from a vector store by semantic similarity, with its match score, and it is these ids that the remediation steps and the narrative cite. Similar past incidents are retrieved alongside, so the agents can reuse a proven resolution rather than improvise one.



Agentic RAG in the open: runbooks and past incidents retrieved by semantic search over a ChromaDB vector store, each with its similarity score, feeding the agents' grounded and cited output.

Together these screens are the loop an operator lives in: detect the anomalies, rank the true cause on the graph, read a narrative that cites its sources, and act on steps grounded in the organisation's own runbooks. Every screen was produced by the running system on real data, with the deterministic engine deciding and the AI layer only explaining what it decided.

How the work was divided

The team froze a shared data contract first, which let all six layers be built and tested in parallel against the same object shapes. Each layer is isolated and single-responsibility, and each was owned end to end by one engineer. What follows is what each person actually built, and the one decision in their layer that mattered most.

Arsh — Data and Ground Truth

Arsh built the evidence base that makes any accuracy claim honest. Rather than assert that the system works, he caused faults deliberately in a realistic environment and recorded exactly what he did and when. He stood up a containerised multi-tier stack that mirrors a real web application, then wrote injection tooling that breaks a single component and stamps the ground truth at the precise moment of failure, giving the whole team a trustworthy answer key to grade every result against.

- A Dockerized multi-tier testbed (load balancer to web to application to database) mirroring a real service.
- Deterministic fault injection using Linux traffic-control, resource squeezes, and configuration changes.
- Ground-truth stamping at injection time: incident id, scenario, exact timestamp, and the true root cause.
- Packet capture and hardware-telemetry recording throughout each injected fault.

Key decision: Stamping ground truth at the moment of injection, not reconstructing it afterwards, so every later accuracy number is credible.

Isha Anomaly Detection

Isha's layer answers not just whether a metric is abnormal, but when it started, which is the raw material causation depends on. She grouped the raw telemetry into per-metric time series and applied change-point detection to recover each anomaly's onset, then measured what counts as normal using robust statistics that the heavy-tailed, spiky nature of real network metrics cannot fool. Her typed anomalies feed the causal engine directly and set the timeline it reasons over.

> Windowing of raw telemetry into per-component, per-metric time series (eight metrics per component).
> PELT change-point detection (rbf kernel, z-score normalised) to recover each fault's exact onset time.
> A robust MAD baseline measured only from the pre-onset segment, so outliers cannot distort it.
> Severity scoring in MAD units, emitting a typed anomaly of component, metric, onset, and severity.

Key decision: Measuring the baseline only on the pre-onset segment, so a sustained fault cannot quietly redefine normal and hide.

Nishant Data Contract and Causal Engine

Nishant owned two layers: the shared data contract the whole team built against, and the causal engine at the product's core. The contract froze thirteen strictly-typed models with machine-checked invariants, so a component cannot claim evidence without citing a raw record, and the same contract generates the frontend's types so the two sides never disagree. The causal engine then turns detected anomalies into a ranked, defensible verdict by reasoning over the dependency graph rather than over timestamps.

> Thirteen frozen Pydantic v2 models with domain invariants, exported to JSON Schema for the frontend.
> The networkx topology twin and topology-constrained candidate filtering that rejects any pathless suspect.
> DoWhy causal attribution (seeded and reproducible) with a deterministic topology-based fallback.
> Counterfactual analysis, the three-tier evidence ledger, and the weighted ranker that sets the final order.

Key decision: Deliberately giving severity the smallest share of the score, so the loudest victim never outranks the quieter true cause.

Shruthika Frontend and NOC Dashboard

Shruthika turned the engine's output into a screen an operator can read under pressure in seconds. The dashboard encodes meaning in colour and shape, so the root cause, the propagation path, and the healthy majority are distinguishable at a glance, and every value on screen is drawn from the real analysed incident rather than a placeholder. She also built a first-class healthy state, so fault-free data produces a clear, explained all-clear instead of an empty screen.

> The Next.js and React NOC dashboard with a Cytoscape topology graph (red root, amber path, green healthy).
> The three-tier evidence ledger interface, the scrubbable incident timeline, and the recommended-actions view.
> The upload-your-own-data flow and a dedicated, explained healthy-state dashboard.
> Per-node telemetry drawers and a typed API client generated from the same frozen contract.

Key decision: Rendering only real data end to end, with no hardcoded numbers, so a judge sees exactly what the engine produced.

Swayam AI Agents, Agentic RAG, API and Audit

Swayam built the layer that explains and verifies the deterministic verdict, serves it over an API, and makes the whole investigation tamper-evident. His three agents are genuinely agentic, using native tool-calling to query the graph, metrics and logs on demand, and to retrieve from a NOC knowledge base so their output is grounded in real playbooks rather than generic model knowledge. They remain strictly bounded: they sharpen the explanation but can never change the ranking, and with no API key the system still returns a complete deterministic report.

- Three bounded agents: an adversarial skeptic, an investigator for the narrative, and a remediation planner.
- Agentic RAG: a ChromaDB vector store over a NOC runbook and past-incident corpus, retrieved by the agents as tools, with every recommendation citing the runbook id it came from.
- A tool-calling runner and contract-validated JSON outputs, with confidence clamped to plus or minus 0.05.
- The FastAPI service (analyze, topology, incidents, replay, audit, knowledge) plus a SHA-256 hash-chained audit trail and an independent verifier.

Key decision: Clamping the AI's influence on confidence, so it can improve the narrative but can never override the cause.

Why isolation mattered

One frozen contract, owned by the data-contract layer, meant detection, the causal engine, the agents, and the frontend could be built simultaneously and still fit together precisely on the first integration. It is the reason a six-person team shipped a single coherent system rather than six disconnected parts.

Results and evaluation

We measure accuracy the honest way, with a synthetic incident benchmark. It injects a known root cause into the real thirty-node topology, lets the fault propagate to its downstream victims, and then asks the engine to recover the cause. The engine sees only the telemetry, logs, alerts, topology, and configuration it would see in production. The injected answer is withheld, so it cannot be read off a label.

Result on the benchmark

Across eight injected scenarios spanning seven fault types (connection-pool exhaustion, bad configuration, capacity exhaustion, DDoS flood, NIC failure, link degradation, and port scan), the engine recovered the correct root cause in every case (accuracy@1), and produced an identical ranking on every run. On the golden reference incident it ranks the database first and rejects the tempting decoys, a config change ten seconds before onset and a cache latency blip, because neither has a dependency path to the symptom.

Why this benchmark is honest

- The engine sees only what it would in production. The injected root cause is never revealed to it, so a correct answer has to be earned from the data.
- Decoys are present by design. A good score requires correctly rejecting plausible-but-false suspects, not just spotting the single obvious anomaly.
- Every stage is deterministic. Baselines, onset detection, and ranking give the same result on every run, so the score reflects the method rather than luck.

What this proves

> The topology constraint works. Suspects with no path to the symptom are floored below every real candidate, which is the difference between causation and correlation.
> The ranking is reproducible. The same incident yields the same answer on every run, because DoWhy is seeded and the fallback is pure.
> The evidence is auditable. Every confirmed item cites a raw record, and the hash-chained audit trail can be independently verified end to end.

Technology and what is next

Layer	Technology
Data contract	Pydantic v2 (frozen models, exported JSON Schema)
Anomaly detection	NumPy / SciPy robust baselines, ruptures PELT change-point
Causal engine	networkx topology twin, DoWhy attribution (seeded, deterministic fallback)
Reasoning agents	OpenAI structured outputs + native tool-calling, bounded confidence
Knowledge / RAG	ChromaDB vector store, OpenAI text-embedding-3-small, runbook + past-incident retrieval
API	FastAPI + Uvicorn
Frontend	Next.js 15, React 19, Cytoscape.js, Recharts, Tailwind, Framer Motion
Audit	SHA-256 hash-chained JSONL ledger

Roadmap

> A rootness tie-break in the ranker for deep non-config faults (the one known edge case).
> An expanded runbook corpus and feedback-driven retrieval ranking for the RAG layer.
> Ingestion adapters for additional real-capture datasets (CSV, PCAP, Parquet) straight into the contract.

In summary

A trustworthy, reproducible root-cause engine that names the true cause, proves it with cited evidence, explains it plainly, and can be independently audited. Built by six people, in six isolated layers, against one frozen contract.